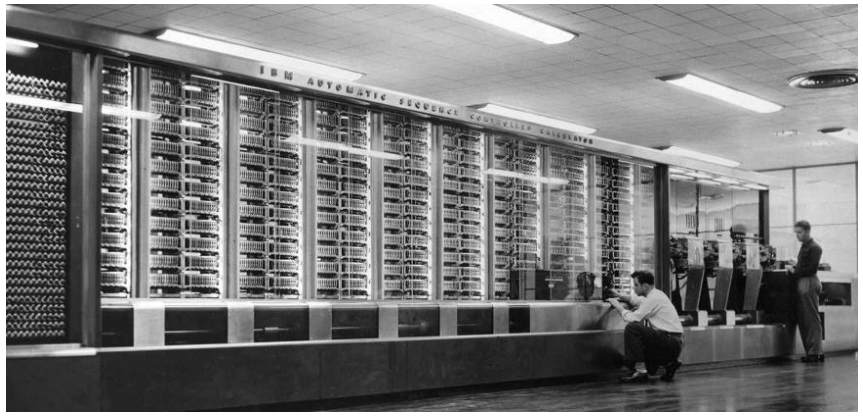
Topic Reviews & Coding Essentials

Grading policy of Programming exams

Some basics

- Function Parameters vs. Arguments
- Declaration vs. Definition
- Function/method declaration --> also called prototype or signature

Bug

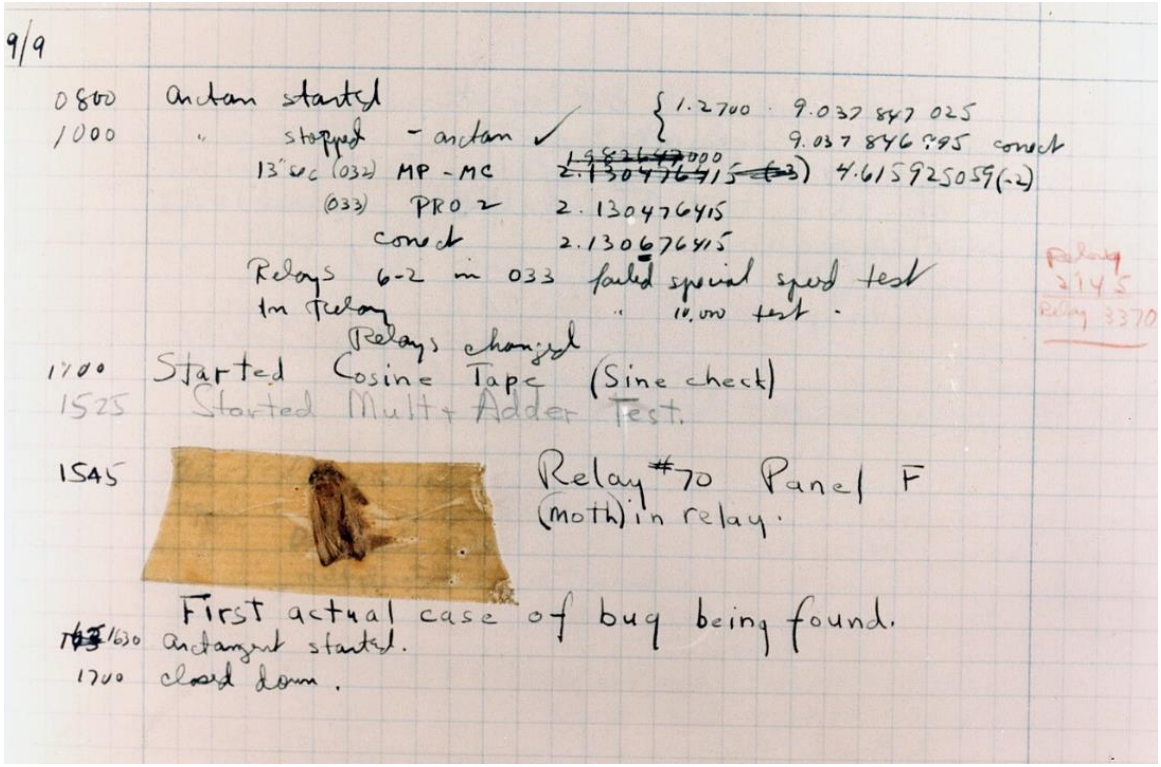


Howard Aiken, Grace Hopper and the Mark I Computer



Hopper in a computer room in Washington, D.C., 1978, photographed by Lynn Gilbert

Grace Hopper - Wikipedia



Bug (engineering) - Wikipedia

Debugging and Memory Leak

OOPs/memory_leak.cpp at main · ahanapradhan/OOPs

Driver program

- OOPs/book.cpp at main · ahanapradhan/OOPs
- DomJudge testcase setup

Inheritance So far...

- Public Inheritance
- Protected members
- Sequence of Ctor/Dtor calls
- Static members
- Upcasting (compile time polymorphism)
- Object Slicing
- Upcasting & Virtual methods (run time polymorphism)
- Abstract base class
- final and override keywords

Copy control of derived objects

OOPs/inheritence_copy.cpp at main · ahanapradhan/OOPs

Diamond Problem

Multiple Inheritance

```
class Animal {
    int id;
    public:
    Animal(int id): id(id) {}
    int getId() {
        cout << id << endl;
        return id;
    }
    virtual void eat(){}
};
class Mammal : public Animal {
    public:
    Mammal(int id): Animal(id) {}
    virtual void breathe(){}
};
class WingedAnimal : public Animal {
    public:
    WingedAnimal(int id): Animal(id) {}
    virtual void flap(){}
};
```

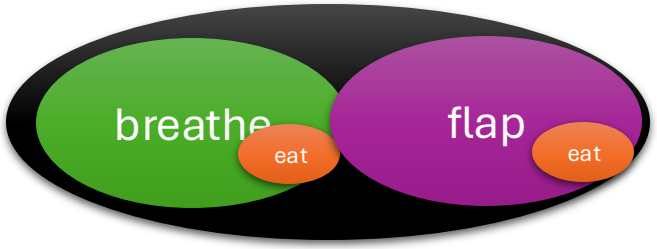




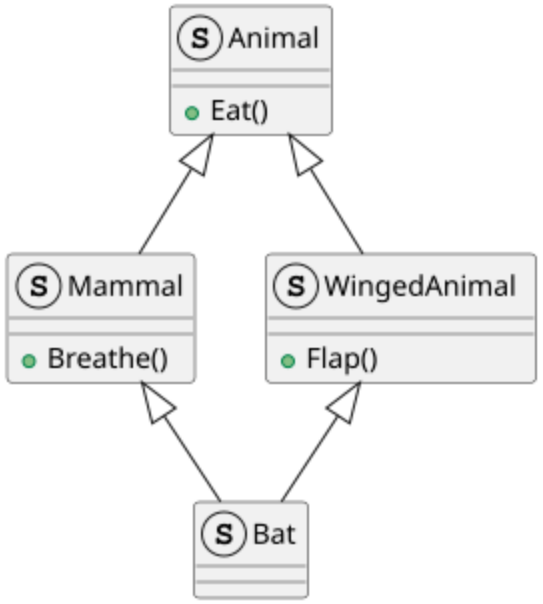



```
class Bat: public Mammal, public WingedAnimal {
    public:
    Bat(int id): Mammal(id), WingedAnimal(++id) {}
};

int main() {
    Bat b( id: 0);
    b.Mammal::getId();
    b.WingedAnimal::getId();
}
```



Multiple Inheritance – Diamond Problem



[Virtual inheritance - Wikipedia]

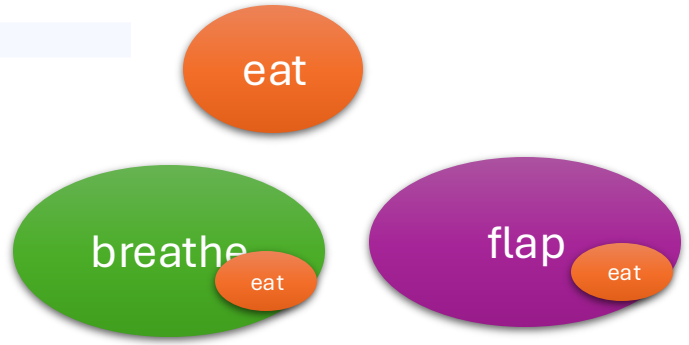
```cpp
class Animal {
public:
    virtual ~Animal() = default;
    virtual void eat() {}
};

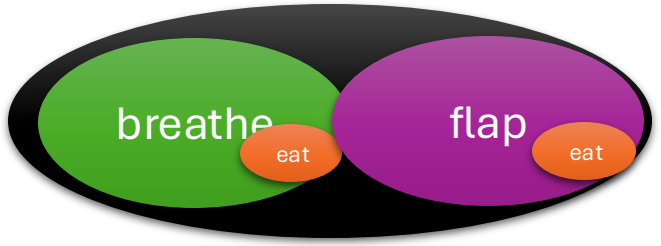
class Mammal: public Animal {
public:
    virtual void breathe() {}
};

class WingedAnimal: public Animal {
public:
    virtual void flap() {}
};

// A bat is a winged mammal
class Bat: public Mammal, public WingedAnimal {};

int main() {
    Bat bat;
    Animal& animal = bat;
    // error: which Animal subobject should a Bat cast into,
    // a Mammal::Animal or a WingedAnimal::Animal?
}
```

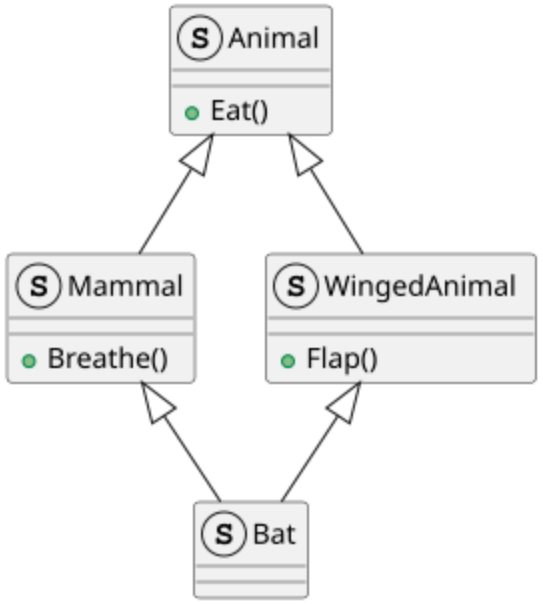




program.cpp C:\Users\ahana\Desktop\solution\for_class 2 problems

Cannot initialize local variable 'animal' of type Animal & with Bat: base class is ambiguous

Multiple Inheritance – Diamond Problem



[Virtual inheritance - Wikipedia]

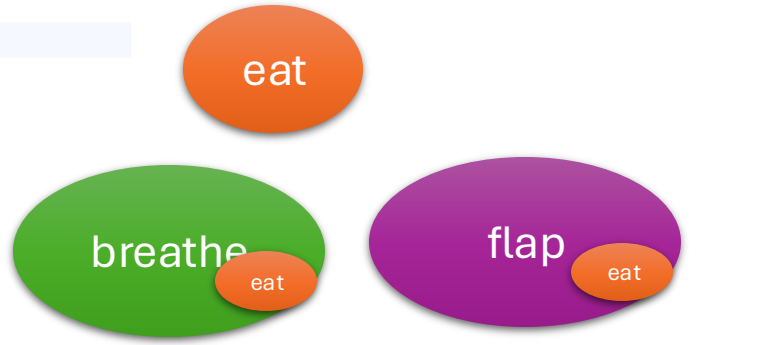
```cpp
class Animal {
public:
    virtual ~Animal() = default;
    virtual void eat() {}
};

class Mammal: public Animal {
public:
    virtual void breathe() {}
};

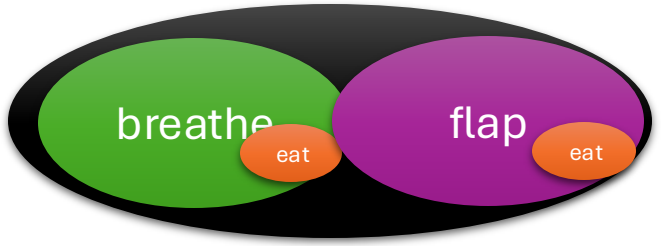
class WingedAnimal: public Animal {
public:
    virtual void flap() {}
};

// A bat is a winged mammal
class Bat: public Mammal, public WingedAnimal {};

int main() {
    Bat bat;
    Animal& animal = bat;
    // error: which Animal subobject should a Bat cast into,
    // a Mammal::Animal or a WingedAnimal::Animal?
}
```



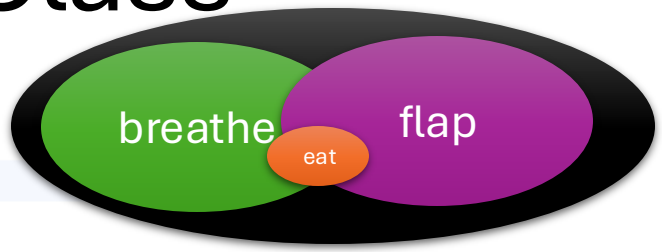

```cpp
Bat bat;
Animal& mammal = static_cast<Mammal&>(bat);
Animal& winged = static_cast<WingedAnimal&>(bat);
```



program.cpp C:\Users\ahana\Desktop\solution\for_class 2 problems

Cannot initialize local variable 'animal' of type Animal & with Bat: base class is ambiguous

Virtual Base Class




```
class Animal {
public:
    virtual ~Animal() = default;
    virtual void eat() {}
};

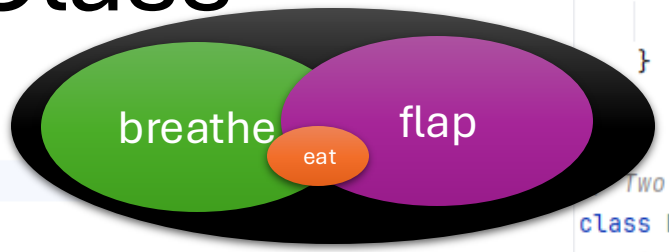
// Two classes virtually inheriting Animal:
class Mammal: virtual public Animal {
public:
    virtual void breathe() {}
};

class WingedAnimal: virtual public Animal {
public:
    virtual void flap() {}
};

// A bat is still a winged mammal
class Bat: public Mammal, public WingedAnimal {};

int main() {
    Bat bat;
    Animal& animal = bat;
}
```

Virtual Base Class




```cpp
class Animal {
public:
    virtual ~Animal() = default;
    virtual void eat() {}
};

// Two classes virtually inheriting Animal:
class Mammal: virtual public Animal {
public:
    virtual void breathe() {}
};

class WingedAnimal: virtual public Animal {
public:
    virtual void flap() {}
};

// A bat is still a winged mammal
class Bat: public Mammal, public WingedAnimal {};

int main() {
    Bat bat;
    Animal& animal = bat;
}
```

```cpp
class Animal {
public:
    virtual ~Animal() = default;
    virtual void eat() {
        cout << "Animal::eat" << endl;
    }

// Two classes virtually inheriting Animal:
class Mammal: virtual public Animal {
public:
    virtual void breathe() {}
    virtual void eat() {
        cout << "Mammal::eat" << endl;
    }
};

class WingedAnimal: virtual public Animal {
public:
    virtual void flap() {}
    virtual void eat() {
        cout << "WingedAnimal::eat" << endl;
    }
};

// A bat is still a winged mammal
class Bat: public Mammal, public WingedAnimal {};

int main() {
    Bat bat;
    Animal& animal = bat;
}
```

Which Override to choose?

Virtual Inheritance and Constructors

```
class Animal {
    int id =0;
    public:
    Animal(int id): id(id) {}
    int getId() {
        cout << id << endl;
        return id;
    }
    virtual void eat(){}
};
class Mammal : virtual public Animal {
    public:
    Mammal(int id): Animal(id) {
        cout<<"Mammal: "<<id<<endl;
    }
    virtual void breathe(){}
};
class WingedAnimal : virtual public Animal {
    public:
    WingedAnimal(int id): Animal(id) {
        cout<<"WingedAnimal: "<<id<<endl;
    }
    virtual void flap(){}
};
class Bat: public Mammal, public WingedAnimal {
    public:
    Bat(int id): Mammal(id), WingedAnimal(++id) {
        cout<<"Bat:" << id <<endl;
    }
};
```

Clangd: Constructor for 'Bat' must explicitly initialize the base class 'Animal' which does not have a default constructor :30

Virtual Inheritance and Constructors

```
int main() {
    Bat b( id: 5);
    b.getId();
}
```

```
class Animal {
    int id =0;
    public:
    Animal(int id): id(id) {}
    int getId() {
        cout << id << endl;
        return id;
    }
    virtual void eat(){}
};
class Mammal : virtual public Animal {
    public:
    Mammal(int id): Animal(id) {
        cout<<"Mammal: "<<id<<endl;
    }
    virtual void breathe(){}
};
class WingedAnimal : virtual public Animal {
    public:
    WingedAnimal(int id): Animal(id) {
        cout<<"WingedAnimal: "<<id<<endl;
    }
    virtual void flap(){}
};
class Bat: public Mammal, public WingedAnimal {
    public:
    Bat(int id): Mammal(id), WingedAnimal(++id) {
        cout<<"Bat:" << id <<endl;
    }
};
```

Clangd: Constructor for 'Bat' must explicitly initialize the base class 'Animal' which does not have a default constructor :30

```
class Animal {
    int id =0;
    public:
    Animal() {
        cout<<"default:"<<endl;
    }
    Animal(int id): id(id) {}
    int getId() {
        cout << id << endl;
        return id;
    }
    virtual void eat(){}
};
class Mammal : virtual public Animal {
    public:
    Mammal(int id): Animal(id) {
        cout<<"Mammal: "<<id<<endl;
    }
    virtual void breathe(){}
};
class WingedAnimal : virtual public Animal {
    public:
    WingedAnimal(int id): Animal(id) {
        cout<<"WingedAnimal: "<<id<<endl;
    }
    virtual void flap(){}
};
class Bat: public Mammal, public WingedAnimal {
    public:
    Bat(int id): Mammal(id), WingedAnimal(++id) {
        cout<<"Bat:" << id <<endl;
    }
};
```

```
C:\Users\ahana\Desktop\s
default:
Mammal: 5
WingedAnimal: 6
Bat:6
0
```

Virtual Inheritance and Constructors

```
int main() {
    Bat b( id: 5);
    b.getId();
}
```

```
class Animal {
    int id =0;
    public:
    Animal(int id): id(id) {}
    int getId() {
        cout << id << endl;
        return id;
    }
    virtual void eat(){}
};
class Mammal : virtual public Animal {
    public:
    Mammal(int id): Animal(id) {
        cout<<"Mammal: "<<id<<endl;
    }
    virtual void breathe(){}
};
class WingedAnimal : virtual public Animal {
    public:
    WingedAnimal(int id): Animal(id) {
        cout<<"WingedAnimal: "<<id<<endl;
    }
    virtual void flap(){}
};
class Bat: public Mammal, public WingedAnimal {
    public:
    Bat(int id): Mammal(id), WingedAnimal(++id) {
        cout<<"Bat:" << id <<endl;
    }
};
```

Clangd: Constructor for 'Bat' must explicitly initialize the base class 'Animal' which does not have a defaul

```
C:\Users\ahana\Deskt
default:
Mammal: 5
WingedAnimal: 6
Bat:6
0
```

```
class Animal {
    int id =0;
    public:
    Animal() {
        cout<<"default:"<<endl;
    }
    Animal(int id): id(id) {}
    int getId() {
        cout << id << endl;
        return id;
    }
    virtual void eat(){}
};
class Mammal : virtual public Animal {
    public:
    Mammal(int id): Animal(id) {
        cout<<"Mammal: "<<id<<endl;
    }
    virtual void breathe(){}
};
class WingedAnimal : virtual public Animal {
    public:
    WingedAnimal(int id): Animal(id) {
        cout<<"WingedAnimal: "<<id<<endl;
    }
    virtual void flap(){}
};
class Bat: public Mammal, public WingedAnimal {
    public:
    Bat(int id): Mammal(id), WingedAnimal(++id) {
        cout<<"Bat:" << id <<endl;
    }
};
```

```
C:\Users\ahana\De
Mammal: 5
WingedAnimal: 6
Bat:6
5
```

```
class Animal {
    int id =0;
    public:
    Animal() {
        cout<<"default:"<<endl;
    }
    Animal(int id): id(id) {}
    int getId() {
        cout << id << endl;
        return id;
    }
    virtual void eat(){}
};
class Mammal : virtual public Animal {
    public:
    Mammal(int id): Animal(id) {
        cout<<"Mammal: "<<id<<endl;
    }
    virtual void breathe(){}
};
class WingedAnimal : virtual public Animal {
    public:
    WingedAnimal(int id): Animal(id) {
        cout<<"WingedAnimal: "<<id<<endl;
    }
    virtual void flap(){}
};
class Bat: public Mammal, public WingedAnimal {
    public:
    Bat(int id): Animal(id), Mammal(id), WingedAnimal(++id) {
        cout<<"Bat:" << id <<endl;
    }
};
```

Templates

Example

```
// swapping of ints
void swap(int &a, int &b) {
    int c;
    c = a;
    a = b;
    b = c;
}
// swapping of floats
void swap(float &a, float &b) {
    float c;
    c = a;
    a = b;
    b = c;
}
// swapping of longs
void swap(long &a, long &b) {
    long c;
    c = a;
    a = b;
    b = c;
}
```

- Overloaded swap( )
  - Parameters are different
  - Bound at compile time
- Inconvenient to maintain 3 versions
  - Any change needs to be copied to all

Function Templates

```
template <class T>
void myswap(T &a, T &b) {
    T c;
    c = a;
    a = b;
    b = c;
}

int main() {
    int x = 10, y = 20;
    float i = 1.5, j = 2.5;
    long u = 10L, v = 20L;

    myswap( a: [&] x,  b: [&] y);
    myswap( a: [&] i,  b: [&] j);
    myswap( a: [&] u,  b: [&] v);

    return 0;
}
```

- Function templates allow defining a generic version of a function with a parameterized type T
- Compiler generates specific versions as needed
- Function declaration must be preceded by
  - keyword "template"
  - "<class Typename>", where Typename is an identifier that is used as a type parameter in the function definition
- Standard type conversions do not apply to function templates (e.g. int to float)
- Function template may be overridden by a specific type, e.g.

  void myswap (int &a, int &b);

Multiple Argument Types

```cpp
#include <iostream>
using namespace std;

template <class T, class U>
void print(T a, U b) {
    cout << a << endl << b << endl;
}

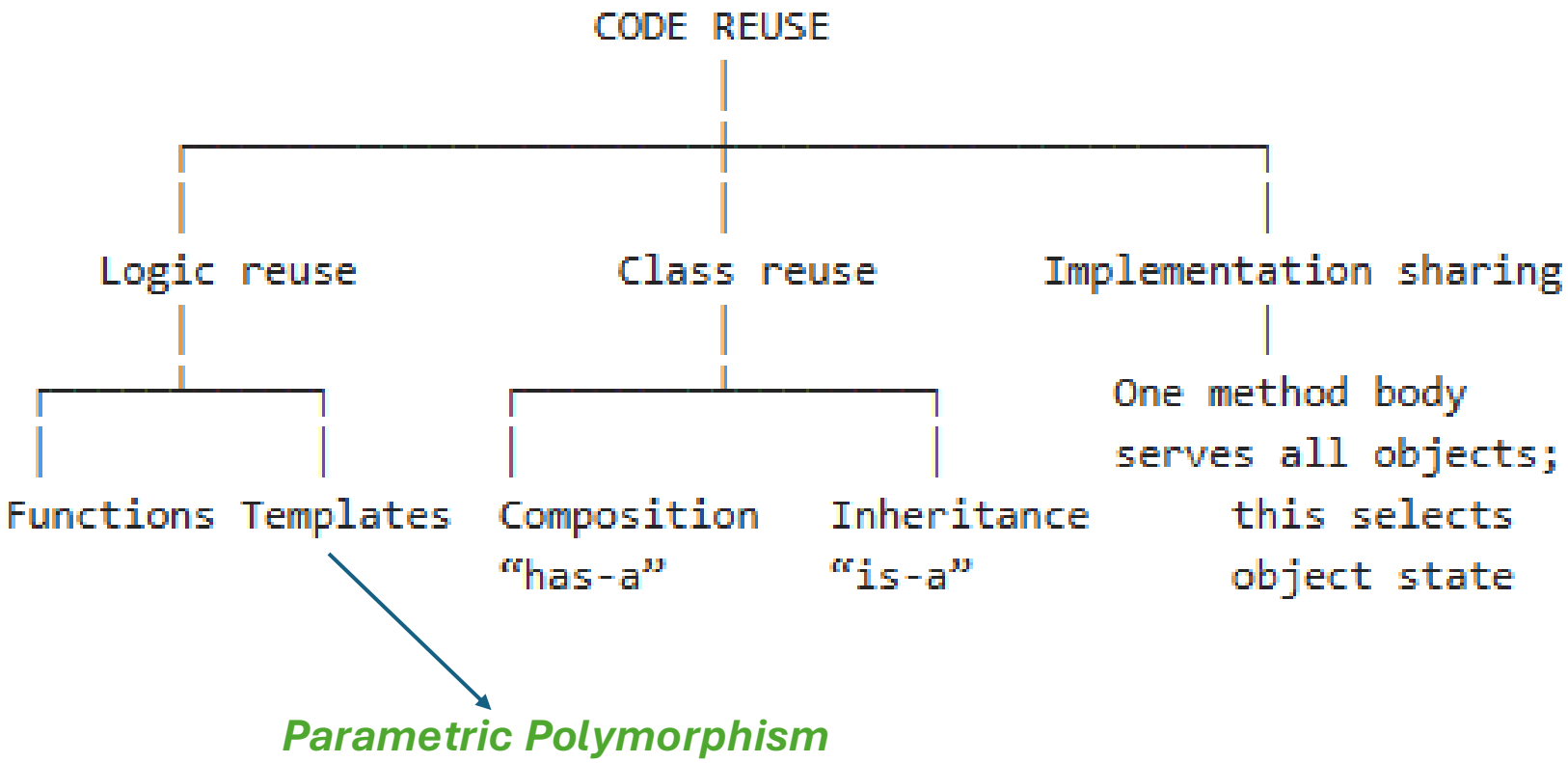
int main() {
    int i = 10;
    float f = 2.5;

    print(i, f);
    print(a: i, b: i);
    print(a: f, b: f);

    return 0;
}
```

- Multiple argument types can be specified if needed, e.g.
  - print mixed types (int / float) in the specified order
- Compiler creates
  - Only as many versions as required by the program

Code Reuse Mechanisms in C++



Function Templates vs. Macros

- Generic functions can also be implemented as macros, e.g.

```
#define min(i, j) ( (i) < (j) ? (i) : (j) )
```

- Template version

```
template <class T>
T min (T i, T j) {
    return (i < j) ? i : j );
}
```

```
#define min(i, j) ((i) < (j) ? (i) : (j))

void macro_check() {
    int a = 5, b = 10;
    int m = min(a++, b);
    // int m = ((a++) < (b) ? (a++) : (b)); -- problem
}
```

- Function templates avoid following problems with Macros
  - Macros are expanded without type checking
  - In the macro version, parameters i and j are evaluated twice – this could give incorrect result if either parameter has a post-incremented variable
  - Compiler error messages with macros refer to the expanded text so debugging is more difficult

Problem Statement

- Implement
  - Stack of Integers
  - Stack of Floats
- Solution
  - IntStack - Class for maintaining stack of Integers
  - FloatStack - Class for maintaining stack of Floats
- Better Solution in C++
  - Stack – Generic class for stack of any type
  - Let compiler generate specific versions of this class

Class Templates

```
const int MAX = 10;

template <class T>
class Stack {
    T stk[MAX];
    int top;

public:
    Stack() {
        top = -1;
    }

    void push(T data) {
        if (top == MAX - 1) {
            cout << "stack is full" << endl;
        } else {
            top++;
            stk[top] = data;
        }
    }
};
```

```
int main() {
    Stack<int> s1;
    s1.push( data: 10);
    s1.push( data: 20);

    Stack<float> s2;
    s2.push( data: 1.1);
    s2.push( data: 2.5);

    return 0;
}
```

Inheritance in Class Templates

```cpp
template <class T>
class Base {
    T data;
public:
    void set(const T& val) {
        data = val;
    }
};

template <class T>
class Derived : public Base<T> {
public:
    void set(const T& v) {
        Base<T>::set(v);
        cout << v << endl;
    }
};
```

```cpp
int main() {
    Derived<int> i;   // creates Base<int> as well
    i.set( v: 10);

    Derived<float> f; // creates Base<float> as well
    f.set( v: 3.14);

    return 0;
}
```

Default Values in Templates

- Function / class template arguments can take default values, e.g.

```
template <class T, int max = 50>
class Stack {
        T arr [ max ];
};
```

- Usage

1. Stack <int, 10> s1; // create an int stack with size of 10
2. Stack <float> s2; // create a float stack with size of 50

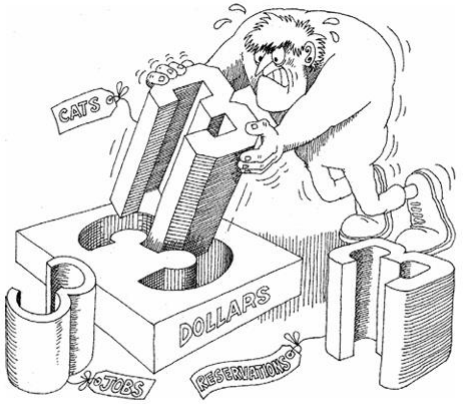
Standard Template Library (STL)

- STL was added to C++ by **Alexander Stepanov**
- Generic and efficient implementation of difficult-to-implement data structures
  - Linked list, Priority queue, Map
- Avoid errors with pointers, e.g. memory leaks, access violations
- Generic algorithms, e.g. sorting
- Generic classes such as containers and iterators
- Save effort on repeated implementations



Alexander Stepanov

Alexander Stepanov - Wikipedia



Strong typing prevents mixing of abstractions.

Image source: Object-Oriented Analysis and Design with Applications by Grady Booch et al.

Containers

- Sequence Containers
  - Represent linear data structures
  - vector, deque (double-ended queue) and list
- Associative Containers
  - Store sets of keys or maps of key-value pairs
  - set, multiset, map and multimap
- Derived Containers
  - Derived from sequential containers
  - stack, queue, priority_queue





```
// Create a stack of strings called cars
stack<string> cars;

// Add elements to the stack
cars.push("Volvo");
cars.push("BMW");
cars.push("Ford");
cars.push("Mazda");
```

C++ Stacks

Iterators

```
// Create a vector called cars that will store strings
vector<string> cars = {"Volvo", "BMW", "Ford", "Mazda"};

// Create a vector iterator called it
vector<string>::iterator it;

// Loop through the vector with the iterator
for (it = cars.begin(); it != cars.end(); ++it) {
  cout << *it << "\n";
}
```

C++ Iterators

- Purpose
  - Go through all the items in a container
- Types
  - Input - Can Read (R), start to end, 1-pass algorithms
  - Output - Can Write (W), start to end, 1-pass algorithms
  - Forward - Can R/W, start to end, 1-pass algorithms
  - Bi-directional - Can R/W, bi-directional, multi-pass algorithms
  - Random - Can R/W, bi-directional, multi-pass algorithms
- Examples
  - vector, deque - random access
  - list, set, multiset, map, multimap - bidirectional
  - stack, queue, priority_queue - no iterators supported

Bidirectional Iterators in C++ - GeeksforGeeks

Summary of Containers

Container	Description
vector	Insert / delete at the end. Direct access to any element via [].
deque	Insert / delete at the beginning / end. Direct access to any element via [].
list	Doubly linked list. Insert / delete anywhere. No direct access to elements.
set	Unique elements. Uses balanced binary search tree for fast lookups.
multiset	Duplicates allowed. Fast lookups like set.
map	Unique key-value pairs. Uses balanced binary search tree for key-based lookups.
multimap	Duplicates allowed. Fast lookups via key like map.
stack	Last in first out list.
queue	First in first out list.
priority_queue	Highest priority element is first element out.

Algorithms

- Algorithms are Template functions
  - Perform common operations
  - Insertion, deletion, searching, sorting
  - Comparing elements or entire containers
- Algorithms
  - Use iterators to access container elements
  - May return results as iterator
- Will an algorithm work on a container?
  - Depends on the iterator types that the container supports
  - All algorithms work on Containers with random-access iterator

vector Class

```
#include <vector>
#include <iostream>
using namespace std ;
int main( ) {
    vector <int> v1 ;
    for ( int i = 0 ; i <= 5 ; ++i )
        v1.push_back ( 5 * i ) ;
    vector <int>::iterator itr1 ;
    for ( itr1 = v1.begin( ) ; itr1 !=
                        v1.end( ); ++itr1 )
        cout << *itr1 << " " ;
    /* prints 0 5 10 15 20 25 */
    vector <int> v2 {10, 20, 30, 40, 50};
    for ( auto itr2 = v2.begin( ) ; itr2 !=
                        v2.end( ) ; ++itr2 )
        cout << *itr2 << " ";
    vector <int> v3 {10, 20, 30, 40, 50};
    for ( auto num : v3 )
        cout << num << " ";
    return 0 ;
}
```

More Vector Operations

```
// Insert element
vector <int>::iterator itr ;
itr = v.begin( ) ;
v.insert ( itr, 25 ) ;

// Read elements
cout << v.front( ) << endl ;
cout << v.back( ) << endl ;
cout << v.at ( 2 ) << endl ;

// Update elements
v[ 2 ] = 20 ;
itr = v.begin( ) ;
*itr = 35 ;
itr += 4 ;
*itr = 99 ;
```

```
// Delete element
itr = v.begin( ) ;
itr += 2 ;
v.erase ( itr ) ;
v.pop_back( ) ;
v.clear( ) ;

// size and capacitiy
cout << v.size( ) << endl ;
cout << v.max_size( ) << endl ;
cout << v.capacity( ) << endl ;
        /* currently allocated capacity
*/
v.resize (100 ) ;
```

Vector of User Defined Point Class

OOPs/vector.cpp at main · ahanapradhan/OOPs

Comparing Vector and C++ array

- Advantage of Vector
  - Can grow / shrink dynamically via `resize()`
- Disadvantage
  - Insertions and deletions are more costly

Linked Lists

OOPs/cpp_list.cpp at main · ahanapradhan/OOPs

Set

- Associative containers
  - Set for unique values
  - Multiset allows duplicates
- Ascending or descending order
- Bi-directional iterators
- Operations supported
  - find an element
  - union
  - intersection
  - difference
  - includes to check if identical

```
#include <set>
int main () {
    set<int> s1 {1, 4, -8, 22, 67, 5};
    set<int> s2 {12, 34, -5, 78, 5};
    set<int>::iterator sitr;
    print_set(s1); // -8, 1, 4, 5, 22, 67
    print_set(s2); // -5, 9, 12, 34, 78

    sitr = find (s1.begin(), s1.end(), 5);
    if (sitr != s1.end()) // found
        cout << “found in s1” << endl;
    else // not found
        cout << “not found in s1” << endl;
    bool b = includes(s1.begin(), s1.end(), s2,begin(),
                s2.end())
    if (b) cout << “sets are identical” << endl;
```

pair and tuple Containers

- std::pair is a basic container for storing elements which naturally occur in pairs e.g.
  - std::map is an abstract container storing keys and their associated values.
  - std::complex is a variant of pair for implementing operations defined on complex numbers.
- std::pair stores two elements, accessed as first and second
- std::tuple generalizes the pair container to a data structure that accommodates any number of different data types
  - std::tuple uses std::get<i> where i is an index starting from 0, giving access to the ordered elements in the tuple.

2D Coordinate
(x, y)

Complex Number Components
(r, i)

3D Coordinate
(x, y, z)

pair Example

```
#include <iostream>
#include <utility>
using namespace std;
main () {
    pair<string, string> student("NAME", "ROLL-NUMBER") ;
    cout << student.first << endl // shows "NAME"
            <<  student.second << endl ; // shows "ROLL-NUMBER"
    // Reassignment using first and second is possible
    student.first =  "FULL-NAME" ;
    student.second = "AADHAR-NUMBER" ;
    student = pair<string, string>("INITIALS", "COURSE-STREAM") ; // Complete
reassignment
}
```

std::array

- Implements a fixed-size array
  - Before using the array container, include <array>
  - To define a std::array, both the data type of its elements and its size must be specified
  - If the data type and/or size differs, we get two different data types, e.g. std::array<Type, M> is different from std::array<Type, N>
  - Array size of 0 is acceptable, even though it is of not much use.
- All its elements are immediately initialized, which makes it more usable than the standard C-style array

std::map

- Offers a (sorted) associative array containing key / value pairs, which may be of any container-accepted type e.g.

  map<string, double> object;

- Before using a map container, the <map> header file must be included
- All keys in a map must be unique
- Multi-map allows duplicate keys
  - [] operator is not supported

```
#include <iostream>
#include <map>
#include <string>
using namespace std;
int main() {
    map<string, int> m1 { {“Rahul”, 765}, {”Disha”, 543},
                {“Neha”, 723}, {“Anuj”, 546} };
    m1 [“Vinesh”] = 345;
    m1 [“Neha”] = 675;
    pair<string, int> p;
    p.first = “Shiva”;
    p.second = 674;
    m1.insert(p);
    map<string, int>::iterator mitr;
    mitr = m1.find(“Disha”);
    if (mitr != m1.end()) cout << ”found” << endl;
}
```

std::stack

- Stack is a sequential container that allows insertion and deletion only at one end
- Last in first out (LIFO) semantics
- Supports push(), pop(), top(), size() and empty() functions

```
#include <iostream>
#include <stack>
using namespace std;

void main() {
    stack<int> stk;

    stk.push(22); stk.push(65);
    stk.push(-2); stk.push(7);
    stk.push(90); stk.push(243);

    int size = stk.size();
    cout << "size is " << size << endl;

    while (!stk.empty() {
        cout << stk.top() << endl;
        stk.pop();
}
```

Queues

- std::queue implements a FIFO data structure
  - implements same functions as std::stack
- std::priority_queue implements a priority queue data structure, which uses priority rules for entry of data
  - Priority queue uses Type::operator< to decide the priority of data elements
  - Smaller the value, lower the priority
- std::deque implements a doubly ended queue data structure

```
#include <iostream>
#include <queue>
using namespace std;

int main () {
   priority_queue<int> pq;

   pq.push(22); pq.push(65);
   pq.push(-2); pq.push(7);
   pq.push(90); pq.push(243);

   int size = pq.size();
   cout << "size is " << size << endl;

   while (!pq.empty() {
         cout << pq.top() << " ";
         pq.pop(); // 243 90 65 22 7 -2
     }
}
```